



Факультет компьютерных наук
НИУ ВШЭ

10 сентября 2026

Проектный семинар «Мобильные колесные роботы»

Дергачев Степан Алексеевич, sadergachev@hse.ru
Муравьев Кирилл Федорович, kmuravev@hse.ru



Содержание курса

1. Инструменты разработки: Linux, Docker, ROS 2.
2. Симуляция мобильного робота: Webots.
3. Кинематика, одометрия, управление и система координат.
4. Датчики, оценка состояния и локализация по известной карте.
5. SLAM и построение карты.
6. Автономная навигация и алгоритмы планирования: ROS2 Nav2
7. Практика на реальных роботах.



Организация курса

Теория

- Знакомство с темой

Практика

- Демонстрации примеров
- Выполнение практических работ

Модуль	Содержание
1	Основы ROS2, модель робота и оценка состояния
2	Локализация, SLAM, навигация, планирование траекторий
3	Практика на реальных роботах



Система оценивания

$$O_{\text{итог}} = 0.25 \cdot O_{\text{экз}} + 0.15 \cdot O_{\text{пз1}} + 0.20 \cdot O_{\text{пз2}} + 0.25 \cdot O_{\text{пз3}} + 0.15 \cdot O_{\text{акт}}$$

Компонент	Обозначение	Вес	Содержание
Экзамен-защита	$O_{\text{экз}}$	25%	Индивидуальная защита результатов после выполнения практических заданий.
Практические задания	$O_{\text{пз1}}$	15%	Решение практических заданий по темам курса
	$O_{\text{пз2}}$	20%	
	$O_{\text{пз3}}$	25%	
Активность	$O_{\text{акт}}$	15%	Активность в выполнении практических заданий на парах, прогресс в выполнении



Выполнение практических работ и активность

Формат

- 1-2 модули – индивидуальное выполнение
- 3 модуль – возможно групповое выполнение

Мягкие дедлайны

- сдача до дедлайна: 100% оценки
- опоздание до 7 дней после: 75% оценки
- опоздание до 14 дней после: 50% оценки



Выполнение практических работ и активность

Оценивание практики. Что нужно сдать:

- Репозиторий на GitHub: код, конфиги, пояснения для воспроизведения результата (README).
- Демонстрационное видео: короткая запись экрана с запуском и наблюдаемым результатом.

Оценивание активности

- Оценивается по наблюдаемому прогрессу на паре.
- Присутствие необходимо, но не достаточно.
- Содержательные вопросы тоже считаются активностью.



Условия автомата

$$O_{\text{акт}} \geq 7.5 \quad \text{и} \quad \frac{0.15 \cdot O_{\text{пз1}} + 0.20 \cdot O_{\text{пз2}} + 0.25 \cdot O_{\text{пз3}}}{0.60} \geq 7.5$$

Экзамен-защита

- Если автомат есть, сдавать экзамен не требуется.
- На экзамене нужно показать результаты практических работ и ответить на вопросы по выполнению.
- Если практическое задание не выполнено, могут быть вопросы по предполагаемому пути выполнения или вопросы по эталонному решению



Командная строка Linux

Общий вид команды

```
[VARIABLE=value] command [flags] [arguments].
```

- `VARIABLE=value` – переменная среды
- `command` – название утилиты/программы/команды
- `flags` – флаги/опции запуска
- `arguments` – аргументы запуска (могут быть обязательными или необязательными, относиться к флагам и нет)

Работа с командной строкой

- Команды работают относительно текущей директории.
- Автодополнение (Tab), история команд (стрелки, `Ctrl+R`) и сочетания клавиш (`Ctrl+A`, `Ctrl+E`, `Ctrl+L` и т.д.) экономят время



Командная строка Linux

Стандартные потоки ввода-вывода

- `stdin` — входные данные (дескриптор: 0)
- `stdout` — обычный вывод (дескриптор: 1)
- `stderr` — сообщения об ошибках (дескриптор: 2)

Перенаправление ввода и вывода

- `command > result.txt` — записать `stdout` в файл
- `command >> result.txt` — дописать `stdout` в конец файла
- `command 2> errors.txt` — записать `stderr` в файл
- `command > all.txt 2>&1` — записать `stdout` и `stderr`
- `command < input.txt` — прочитать `stdin` из файла
- `command1 | command2` — передать `stdout` следующей команде
- `command | tee result.txt` — вывести на экран и записать в файл
- `command > /dev/null` — отбросить вывод



Командная строка Linux

Навигация и файлы

- `pwd, cd, ls, tree`
- `mkdir, touch, cp, mv, rm`
- `cat, less, head, tail, nano`
- `which, find`

ОС, процессы и диагностика

- `ps, htop, pgrep, kill, pkill,`
- `grep,`
- `chmod, sudo, whoami, id`
- `echo $VARIABLE, env, export VARIABLE=10`



SSH и удалённая разработка

- SSH (Secure Shell) – сетевой протокол, который позволяет управлять компьютерами удаленно через командную строку.
- Зачастую необходим в робототехнике при подключении к бортовому компьютеру работа
- VS Code (с набором расширений Remote Development) позволяет открыть удалённую папку почти как локальный проект.

Примеры:

- `ssh user@host_address` – подключение к удаленному компьютеру
- `scp /local-path/file user@host_address:/remote-path/` – загрузить файл на удаленный компьютер
- `scp user@host_address:/remote-path/file /local-path/` – скачать файл с удаленного компьютера



Docker

Docker позволяет запускать приложения в изолированном и воспроизводимом окружении.

Образ

- Шаблон окружения.
- Собирается из `Dockerfile`
- Содержит образ ОС, установленные пакеты и загруженные файлы.

Контейнер

- Запущенный экземпляр образа
- Включает запущенные процессы, изолированные от основной среды
- Может использовать сеть и примонтированные тома (volumes) хост-системы



Docker Compose

Docker Compose позволяет управлять и запускать контейнеры (или группу контейнеров) с помощью одного конфигурационного YAML-файла

```
1 services:
2     ros2-base:
3         build:
4             context: .
5             dockerfile: docker/Dockerfile
6         image: mobile-robots:jazzy
7         volumes:
8             - ./src:/home/devuser/robot_ws/src
9         environment:
10             DISPLAY: ${DISPLAY}
```



Docker: Примеры команд

Docker

- `docker images`
- `docker pull`
- `docker build`
- `docker run`
- `docker ps`
- `docker exec`
- `docker start`
- `docker stop`
- `docker rm`

Docker Compose

- `docker compose build`
- `docker compose up`
- `docker compose ps`
- `docker compose exec`
- `docker compose stop`
- `docker compose down`



Зачем нам Docker и Docker Compose

Docker обеспечивает:

- Всегда фиксированные версии библиотек и пакетов
- Изоляцию зависимостей от основной системы
- Хранение конфигурации рядом с исходным кодом в Git
- Воспроизводимую сборку рабочего окружения



VS Code: Полезные расширения

- Remote Development Pack, Docker Extension Pack, Docker DX – удаленная разработка
- Python, Pylance, Python Environments, Python Debugger, Black Formatter, Jupyter Extension – разработка на Python
- C/C++ Extension Pack, clangd, Clang-Format – разработка на C++
- Markdown, XML, YAML, Edit CSV и др. – поддержка языков разметки и различных типов файлов.



Robot Operating System

Robot Operating System (ROS) – это набор библиотек, инструментов и соглашений для сборки робототехнических систем из взаимодействующих процессов.

- Разделение системы на независимые компоненты: драйверы, обработка данных с датчиков, SLAM, планирование, управление.
- Соглашение о форматах сообщений и интерфейсах между компонентами.
- Механизмы запуска, наблюдения, записи и воспроизведения поведения системы.
- Доступность готовых пакетов.





ROS 1 и ROS 2

	ROS 1	ROS 2
Связь процессов	Централизованная, требуется запуск <code>roscore</code>	Децентрализованная
Реальное время	Ограниченная поддержка	Потенциал есть, нужна настройка
Сеть	Собственный стек на основе TCP/UDP. Проще настраивается.	Протокол DDS, большая вариативность, но сложная настройка
Состояние	Больше не поддерживается	Активно развивается



Система версий ROS и Ubuntu

- ROS 2 распространяется дистрибутивами: Humble, Iron, Jazzy, Kilted и т.д.
- Дистрибутивы выходят раз в год, каждые два года – LTS релиз
- Версия ROS привязана к LTS версиям Ubuntu

Distro	Release date	Logo	EOL date	ROS Boss
Lyrical Luth	May 22, 2026		May 2031	Shane Loretz
Kilted Kaiju	May 23, 2025		December 2026	Scott K. Logan
Jazzy Jalisco	May 23, 2024		May 2029	Marco A. Gutiérrez
Iron Iwini	May 23, 2023		December 4, 2024	Yadunund Vijay

Version (+ more info)	Ubuntu Resolute	Ubuntu Noble	Ubuntu Jammy	Windows 11	Windows 10	MacOS
Lyrical Luth	+	•	×	+	×	•
Kilted Kaiju	×	+	×	×	***	•
Jazzy Jalisco	×	+	×	×	***	•
Humble Hawksbill	×	×	+	×	***	•
Rolling Ridley	+	×	×	+	×	•

⚡ Tier 1: Fully Supported & Recommended for New Users *** Tier 1: Fully Supported ⚡ Tier 2: Limited Support • Tier 3: Community Support



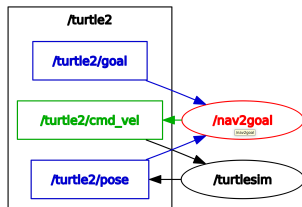
ROS 2: Основные концепции

- **Node (узел)** – процесс или компонент, который выполняет какую-либо задачу.
- **Topic** – механизм передачи сообщений между узлами через именованный канал.
- **Service** – механизм передачи запроса и ответа между узлами для выполнения коротких операций.
- **Action** – механизм передачи запроса и ответов о ходе выполнения долгих задач с обратной связью.
- **Parameters** – механизм настройки узлов во время запуска или выполнения.
- **Launch-файлы** – конфигурационные/скриптовые файлы с описанием того, что и как запустить вместе.



ROS2 Topic

- Publisher публикует сообщения, не ожидая ответа.
- Subscriber получает сообщения через функцию-callback.
- Один topic может иметь несколько publishers и subscribers.
- Все участники должны использовать совместимый тип сообщения.





ROS2 Topic

Примеры топиков и типов сообщений:

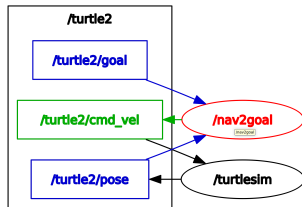
`/turtle1/cmd_vel -> geometry_msgs/msg/Twist`
`/turtle1/pose -> turtlesim/msg/Pose`

Примеры диагностики:

```
ros2 topic list
ros2 topic type /turtle1/pose
ros2 topic info /turtle1/pose
ros2 topic echo /turtle1/pose
ros2 topic hz /turtle1/pose
ros2 interface show geometry_msgs/msg/Twist
```

Публикация:

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
'linear: x: 0.0 y: 0.0 z: 0.0 angular: x: 0.0 y: 0.0 z: 0.0'
```





ROS2 service и action

Service

- Клиент отправляет запрос, сервер выполняет короткую операцию и возвращает ответ

Action

- Клиент отправляет цель и может получать информацию о ходе выполнения.

Примеры:

```
ros2 service list
ros2 service type /spawn
ros2 service info /spawn
```

```
ros2 action list
ros2 action type /turtle1/rotate_absolute
ros2 action info /turtle1/rotate_absolute
```



Инструменты ROS 2 для диагностики

- CLI интерфейс: `ros2 node ...`, `ros2 topic ...`, `ros2 service ...`, `ros2 action ...`
- Граф связей между узлами: `rqt_graph`
- Запись и воспроизведение данных: `ros2 bag`
- Инструменты визуализации `rviz` и `Foxglove`
- Логи узлов в топиках `/rosout` и `/diagnostics`



ROS 2 Workspace и ROS2 пакеты

- Пакет является минимальной единицей организации кода в ROS.
- ROS 2 Workspace – рабочая директория, где хранятся ваши пакеты
- Установка зависимостей утилитой `rosdep`, сборка пакетов утилитой `colcon`:

```
colcon build --symlink-install
```

- Структура workspace:
 - `src/`: исходный код пакетов.
 - `build/`: промежуточные файлы сборки.
 - `install/`: результат сборки.
 - `log/`: логи сборки.

Инициализация окружения

```
source /opt/ros/jazzy/setup.zsh  
source install/setup.zsh
```



Подготовка к практическому занятию

- Установить Linux, например Ubuntu 24.04 (на Mac необходимо использовать виртуальную машину UTM)
- Установить или проверить Docker Engine в Linux.
- Собрать и запустить контейнер с окружением курса, проверить запуск команд ROS2, GUI приложений
- Установить VS Code и нужные расширения, подключиться к контейнеру.



PathPlanning/2026-Project-Seminar-Mobile-
Wheeled-Robots



Спасибо за внимание!